Atividade 04: Controle de LED por botão

Discente: Andreza costa dos Santos

Objetivo:

Elaborar um controlador do estado de LEDs por meio de botão. Esta atividade será realizada no simulador Wokwi e deve utilizar a ESP-IDF como framework de desenvolvimento.

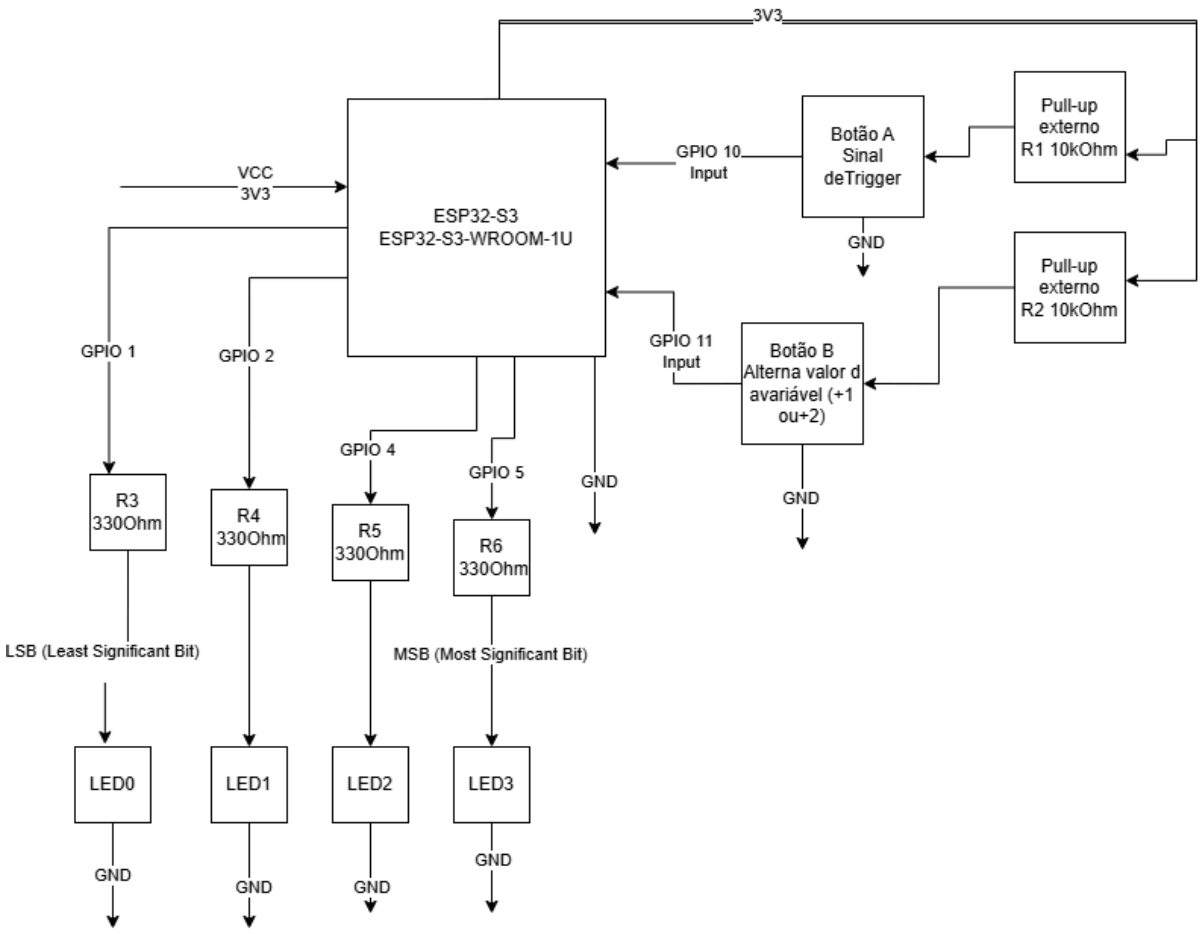
Material Necessário:

- ESP32S3
- 4 LED
- 2 botões
- Outros componentes para implementar os drivers
- Conta no Wokwi (https://wokwi.com/)

Passos para a Atividade:

1. **Elaborar o diagrama em bloco de um contador binário de 4 bits com dois botões de controle com os seguinte componentes:**
    - ESP32S3;
    - 4 LED;
    - 2 Buttons;
    - Componentes para os circuitos de driver.

RESPOSTA:

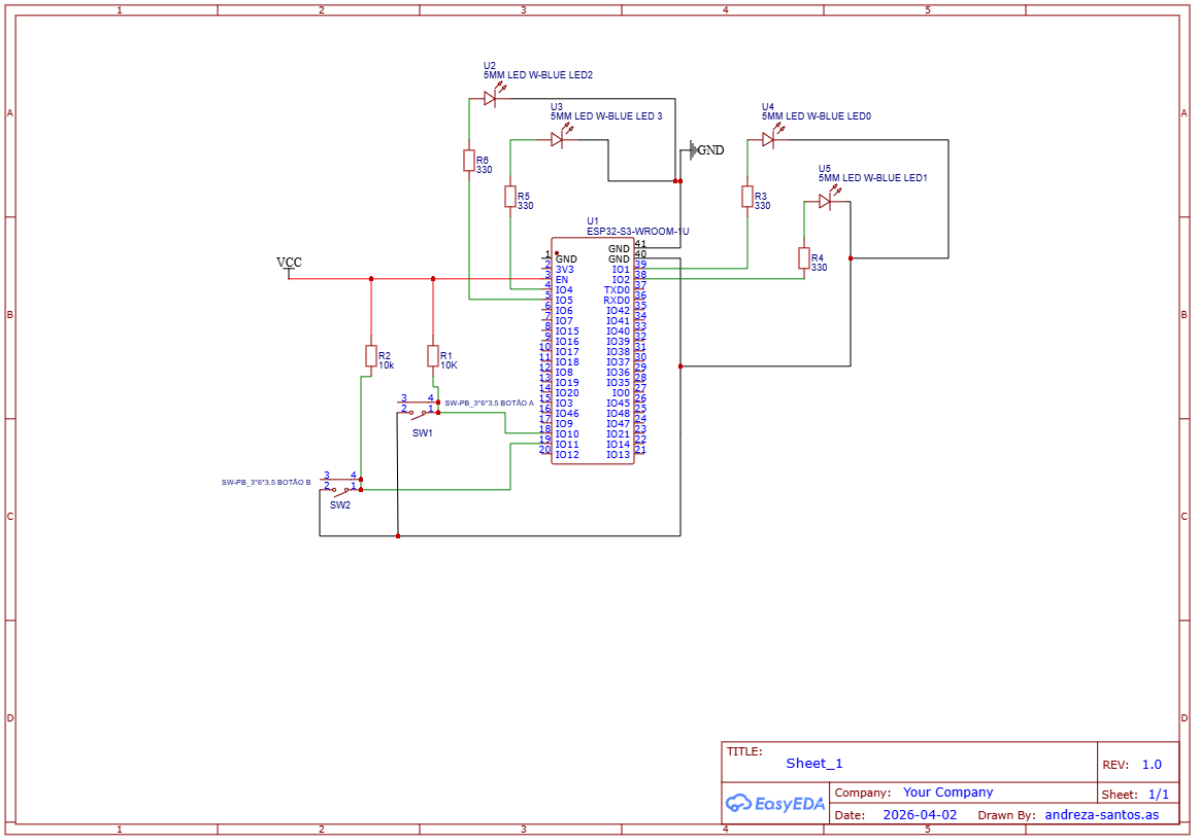


Grupo	Componente	Origem (Pino/Barramento)	Destino (Pino ESP32-S3 / GND)	Função Técnica
Alimentação	ESP32-S3 VCC	Barramento 3.3V	**Pino 2 (3V3)**	Alimentação principal do chip
Alimentação	ESP32-S3 GND	Barramento GND	**Pino 1, 40 ou 41 (GND)**	Referência negativa comum
Alimentação	ESP32-S3 EN	**Pino 2 (3V3)**	**Pino 3 (EN)**	Habilita a operação do chip
---	---	---	---	---

Entrada	Resistor R1 (10k)	Barramento 3.3V	**Pino 18 (IO10)**	Pull-up externo do Botão A
Entrada	Botão A (SW1)	**Pino 18 (IO10)**	**GND**	Incremento do contador
Entrada	Resistor R2 (10k)	Barramento 3.3V	**Pino 19 (IO11)**	Pull-up externo do Botão B
Entrada	Botão B (SW2)	**Pino 19 (IO11)**	**GND**	Alterna Passo (+1 ou +2)
---	---	---	---	---
Saída	LED 0 (LSB)	**Pino 39 (IO1)**	Via Resistor R3 (330R) p/ **GND**	Bit 2^0(Peso 1)
Saída	LED 1	**Pino 38 (IO2)**	Via Resistor R4 (330R) p/ **GND**	Bit 2^1 (Peso 2)
Saída	LED 2	**Pino 4 (IO4)**	Via Resistor R5 (330R) p/ **GND**	Bit 2^2 (Peso 4)
Saída	LED 3 (MSB)	**Pino 5 (IO5)**	Via Resistor R6 (330R) p/ **GND**	Bit 2^3 (Peso 8)

2. **Elaborar o esquemático para o contador.**

RESPOSTA:



3. **Desenvolvimento do Código:**

Desenvolva um programa utilizando o ESP-IDF para implementar um contador binário de 4 bits. O valor atual do contador deve ser exibido utilizando 4 LEDs. Além disso, o sistema deve utilizar 2 botões conectados a entradas digitais, com as seguintes funcionalidades:

- **Botão A**: a cada acionamento, deve incrementar o valor do contador conforme a unidade de incremento atual (padrão: +1).

- **Botão B**: alterna a unidade de incremento entre **+1 e +2** a cada acionamento.

Obs:

- O contador deve circular (isto é, ao ultrapassar o valor máximo de 4 bits, ele retorna ao início com base no passo atual).
- O contador é de 4 bits, portanto seu valor varia entre `0x0` (0 decimal ou b0000) e `0xF` (15 decimal ou b1111).
- A unidade de incremento inicial é 1 unidade.
- Ao acionar o Botão B, a unidade de incremento muda para 2 unidades. Um novo acionamento alterna novamente para 1 unidade, e assim por diante.

- A lógica de estouro (overflow) deve considerar a unidade de incremento:
  - Se o contador estiver em `0xF` e a unidade de incremento for 1, o próximo valor será `0x0`.
  - Se estiver em `0xE` (14 decimal), o próximo valor com incremento de 2 será `0x0`.
  - Se estiver em `0xF` (15 decimal) e incremento de 2, o resultado será `0x1`.
- O valor final do contador deve sempre estar dentro do intervalo de 4 bits (`0x0` a `0xF`) após cada incremento.
- O debounce deve ser tratado por software (Não usar daley).

RESPOSTA:

código:

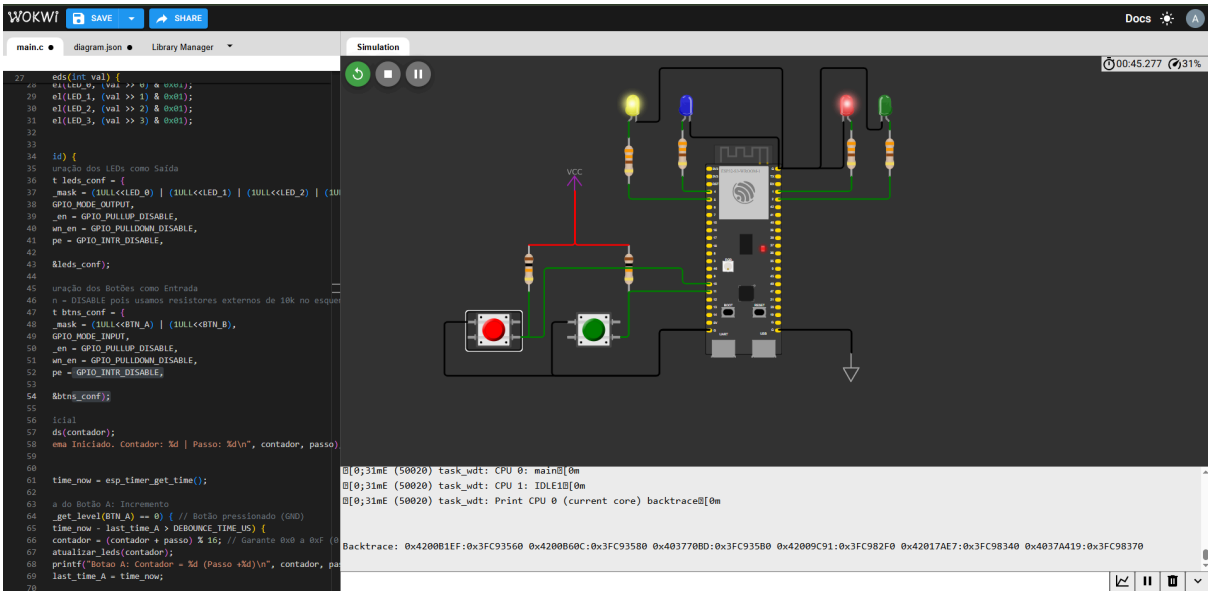
```
#include <stdio.h>

#include "driver/gpio.h"

#include "esp_timer.h"



// Mapeamento de pinos conforme seu esquemático

#define LED_0 1   // LSB (Bit 0)

#define LED_1 2   // Bit 1

#define LED_2 4   // Bit 2

#define LED_3 5   // MSB (Bit 3)

#define BTN_A 10  // Botão de Incremento

#define BTN_B 11  // Botão de Alternar Passo



// Tempo de debounce em microssegundos (50ms)

#define DEBOUNCE_TIME_US 50000



// Variáveis de estado do sistema

int contador = 0;
```

```
int passo = 1;


// Variáveis para controle de tempo (Debounce sem delay)

int64_t last_time_A = 0;

int64_t last_time_B = 0;


/**

 * Atualiza os 4 LEDs com base no valor binário do contador

 */

void atualizar_leds(int val) {

    gpio_set_level(LED_0, (val >> 0) & 0x01);

    gpio_set_level(LED_1, (val >> 1) & 0x01);

    gpio_set_level(LED_2, (val >> 2) & 0x01);

    gpio_set_level(LED_3, (val >> 3) & 0x01);

}


void app_main(void) {

    // 1. Configuração dos LEDs como Saída

    gpio_config_t leds_conf = {

        .pin_bit_mask = (1ULL<<LED_0) | (1ULL<<LED_1) | (1ULL<<LED_2) |
(1ULL<<LED_3),

        .mode = GPIO_MODE_OUTPUT,

        .pull_up_en = GPIO_PULLUP_DISABLE,

        .pull_down_en = GPIO_PULLDOWN_DISABLE,

        .intr_type = GPIO_INTR_DISABLE,
```

```
    };
    gpio_config(&leds_conf);

    // 2. Configuração dos Botões como Entrada
    // pull_up_en = DISABLE pois usamos resistores externos de 10k no
esquema
    gpio_config_t btns_conf = {
        .pin_bit_mask = (1ULL<<BTN_A) | (1ULL<<BTN_B),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE,
    };
    gpio_config(&btns_conf);

    // Estado inicial
    atualizar_leds(contador);
    printf("Sistema Iniciado. Contador: %d | Passo: %d\n", contador,
passo);

    while (1) {
        int64_t time_now = esp_timer_get_time();

        // Lógica do Botão A: Incremento
        if (gpio_get_level(BTN_A) == 0) { // Botão pressionado (GND)
            if (time_now - last_time_A > DEBOUNCE_TIME_US) {
```

```
                contador = (contador + passo) % 16; // Garante 0x0 a
0xF (0 a 15)

                atualizar_leds(contador);

                printf("Botao A: Contador = %d (Passo +%d)\n",
contador, passo);

                last_time_A = time_now;

            }

        }



        // Lógica do Botão B: Alternar Passo (+1 ou +2)

        if (gpio_get_level(BTN_B) == 0) {

            if (time_now - last_time_B > DEBOUNCE_TIME_US) {

                passo = (passo == 1) ? 2 : 1;

                printf("Botao B: Unidade de incremento alterada para:
%d\n", passo);

                last_time_B = time_now;

            }

        }

    }

}
```

PRINT DA SIMULAÇÃO:



LINK DE ACESSO À SIMULAÇÃO: https://wokwi.com/projects/460228638135865345